Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

INTEGRATED LINUX STORAGE INFRASTRUCTURE

Submitted by

PRUTHVIRAJ M (2411021061330)

MARIYANI VISHAL (2411021061591)

GANESH KOTI (2411021061650)

ROHAN PUJERI (2411021061648)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Prof. Ananthanagu U

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Oct 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled “Integrated Linux Storage Infrastructure” submitted by Pruthviraj M [2411021061330], Mariyani Vishal [2411021061591], Ganesh Koti [2411021061650] and Rohan Pujeri [2411021061648] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Alliance University, is a bonafide work during the academic year 2026-2027.

Prof. Kusuma J	**Prof. Ananthanagu U**	**Dr. Shekhar R**
Course Handling Faculty	Course Coordinator	Professor & HoD

Table of Contents

1. Introduction

1.1 Aim

1.2 Background

1.3 Problem Statement

1.4 Objectives

2. Methodology

2.1 Tools and Technologies Used

2.2 Approach / Development Process

3. Linux System Administration – Design and Implementation

3.1 System Architecture

3.2 Linux Environment Setup

3.3 User and Group Management

3.4 File and Directory Management

3.5 Package and Process Management

3.6 Storage and File System Configuration

3.7 Backup, Automation and Monitoring

3.8 Menu-Driven Interface

3.9 Security and Access Control

4. Results and Testing

4.1 Implementation Results

4.2 Testing and Validation

4.3 Challenges Faced and Solutions

5. Conclusion and Future Scope

References

Appendices

1. Introduction

1.1 Aim

The aim of this project is to design, build and test an Integrated Linux Storage Infrastructure on Ubuntu. The system divides a disk into partitions, manages the space with LVM (Logical Volume Manager), creates and mounts filesystems, takes automatic backups, monitors disk usage and gives a menu-driven Bash interface to operate everything.

The whole project uses only Bash shell scripting and standard Linux commands.

1.2 Background

Linux is widely used in servers and cloud systems. Storage management is an important part of Linux system administration. Normal disk partitions have a fixed size, so it is hard to increase the space when a disk becomes full. If there is no backup, data can be lost, and if nobody monitors the disk, a full disk goes unnoticed until applications fail.

LVM solves the size problem, because a logical volume can be extended while it is in use. A backup script protects the data, cron runs tasks automatically, and a monitoring script warns the administrator when usage is high.

1.3 Problem Statement

On many Linux systems, storage is set up in a fixed way. A full disk cannot be extended easily, data can be lost without a backup, manual backups are forgotten, and nobody notices when the disk is almost full. This project builds one complete storage system that can grow, is backed up automatically, and warns the user before the space runs out.

1.4 Objectives

The main objectives of the project are:

- To create and manage disk partitions using parted.
- To build flexible storage using LVM (PV, VG and LV).
- To create ext4 filesystems and mount them permanently using /etc/fstab.
- To extend a volume online and to take a snapshot.
- To implement an automated backup with a tested restore.

- To automate tasks using Bash scripts and cron.
- To monitor disk usage and write a warning above 80%.
- To provide a menu-driven Bash interface with input checking and error handling.
- To test and document the complete infrastructure.

2. Methodology

2.1 Tools and Technologies Used

Software Requirements

Software / Tool	Purpose
Ubuntu 25.04 (Plucky Puffin), 64-bit	Operating system
Oracle VirtualBox	Virtual machine with a virtual 10 GB extra disk
parted	Creating the GPT table and partitions
lvm2 (pvcreate, vgcreate, lvcreate, lvextend)	Flexible storage management
ext4 and /etc/fstab	Filesystem and permanent mounting
tar	Compressed backups and restore
Bash scripting	Backup, monitoring and menu scripts
cron	Scheduling scripts automatically
df, lsblk, pvs, vgs, lvs	Checking and monitoring storage
sed, nano	Editing scripts and configuration files

Hardware Requirements

Item	Details
Host computer	PC or laptop with virtualization enabled
Memory for the VM	4 GB RAM
Processor for the VM	2 CPU cores
System disk (sda)	25 GB virtual disk
Extra disk (sdb)	10 GB virtual disk used for the project

Other Tools and Resources

A screenshot tool was used to capture the output of each step, and Microsoft Word and PowerPoint were used for the report and the presentation.

2.2 Approach / Development Process

The project was done in the following phases:

1. Linux setup: Ubuntu in VirtualBox and installation of the required tools.
2. Partitions: three partitions on /dev/sdb using parted.
3. LVM: physical volumes, volume group and logical volumes.
4. Filesystem and mounting: ext4 filesystems and permanent mounts using fstab.
5. LVM management: extending a volume online and taking a snapshot.
6. Backup: a tar backup script and a restore test.
7. Automation: cron jobs for backup and monitoring.
8. Monitoring: a script that writes a warning above 80% usage.
9. Menu-driven interface: one Bash script that operates the whole system.
10. Testing and documentation: tests, screenshots and this report.

3. Linux System Administration – Design and Implementation

3.1 System Architecture

The storage is built in layers. The raw disk is divided into partitions, the partitions become LVM physical volumes, the volumes are pooled into one volume group, and logical volumes are created from the pool. Each logical volume gets an ext4 filesystem and is mounted on a folder. Backup, cron and monitoring work on top of these mounted folders.

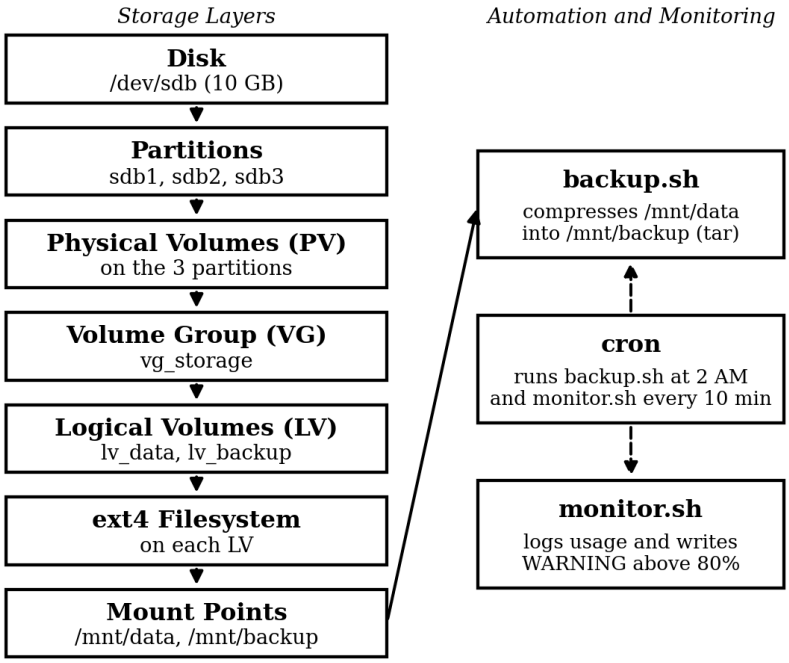


Figure 3.1: System architecture of the Integrated Linux Storage Infrastructure

The main components are:

1. **Disk:** /dev/sdb, a 10 GB virtual disk.
2. **Partitions:** sdb1 (4 GB), sdb2 (4 GB) and sdb3 (2 GB).
3. **LVM:** volume group vg_storage with logical volumes lv_data and lv_backup.
4. **Mount points:** /mnt/data for files and /mnt/backup for backups.
5. **Backup and automation:** backup.sh and cron.
6. **Monitoring:** monitor.sh writes usage and warnings to a log file.

7. **Interface:** storage_manager.sh, the menu-driven Bash script.

3.2 Linux Environment Setup

3.2.1 Virtual Machine

Ubuntu 25.04 was installed in Oracle VirtualBox with 4 GB RAM and 2 CPU cores. The VM has a 25 GB system disk (sda) and one extra 10 GB virtual disk (sdb). The extra disk is empty and is the only disk used for the storage project, so the system disk is never changed.

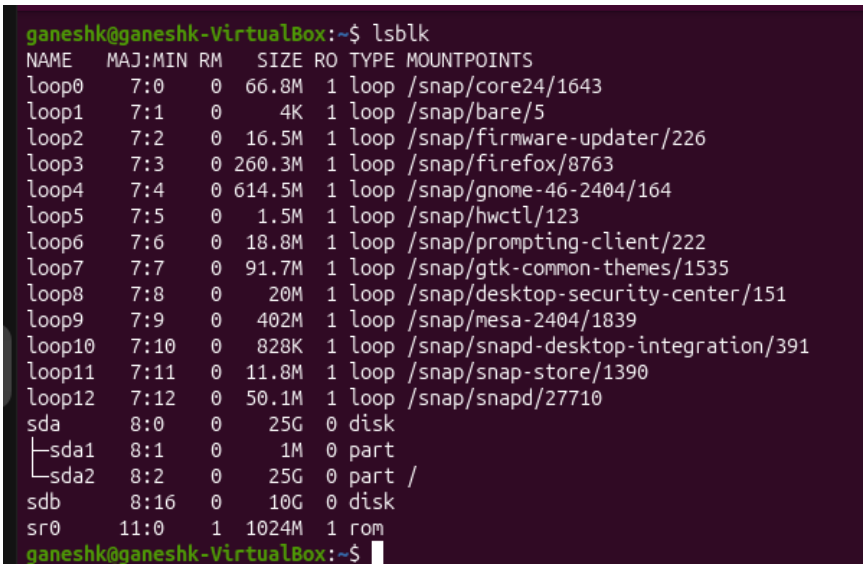


Figure 3.2: Disks before configuration: sda (system) and sdb (empty extra disk)

3.2.2 Installing the Required Tools

The package list was updated and the required tools were installed:

```
sudo apt update
sudo apt install -y lvm2 parted xfsprogs rsync sysstat smartmontools
```

The installation was checked with the commands lvm version and parted --version.

3.3 User and Group Management

All storage tasks need administrator rights, so they were run by the normal Ubuntu user with sudo. No extra user accounts were required for this project. The backup files created in /mnt/backup are owned by root, which protects them from changes by normal users. The menu script also checks that it is started with sudo and refuses to run otherwise.

```
sudo storage_manager.sh      (runs the menu with administrator rights)
ls -l /mnt/backup            (shows the owner of the backup files)
```

3.4 File and Directory Management

The following folders and files were created and used:

Path	Purpose
/mnt/data	Mount point of lv_data (working files)
/mnt/backup	Mount point of lv_backup (backup archives)
/mnt/snap	Temporary mount point to view a snapshot
~/restore_test or /tmp/restore_test	Folder used to test the restore
/usr/local/bin	Location of backup.sh, monitor.sh and storage_manager.sh
/var/log	backup.log and storage_monitor.log

Test files test.txt, notes.txt and record.txt were created in /mnt/data using echo, and the folders were created with mkdir -p.

3.5 Package and Process Management

Packages were installed with apt. The cron service runs in the background and starts the scheduled scripts. Its status was checked with systemctl and it is active (running) and enabled, so it starts automatically after a reboot.

```
sudo systemctl status cron
```

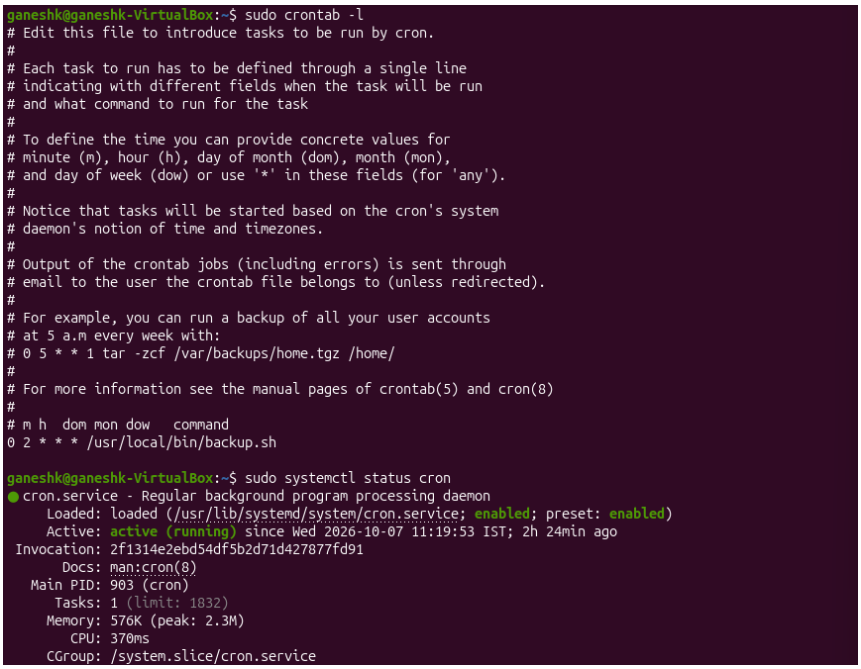


Figure 3.3: Cron jobs (crontab -l) and the cron service status

3.6 Storage and File System Configuration

3.6.1 Partitioning

A GPT partition table was created on /dev/sdb and three partitions were made. All three were marked with the lvm flag so that LVM can use them.

```
sudo parted /dev/sdb --script mklabel gpt
sudo parted /dev/sdb --script mkpart primary 1MiB 4GiB
sudo parted /dev/sdb --script mkpart primary 4GiB 8GiB
sudo parted /dev/sdb --script mkpart primary 8GiB 100%
sudo parted /dev/sdb --script set 1 lvm on
sudo parted /dev/sdb --script set 2 lvm on
sudo parted /dev/sdb --script set 3 lvm on
```

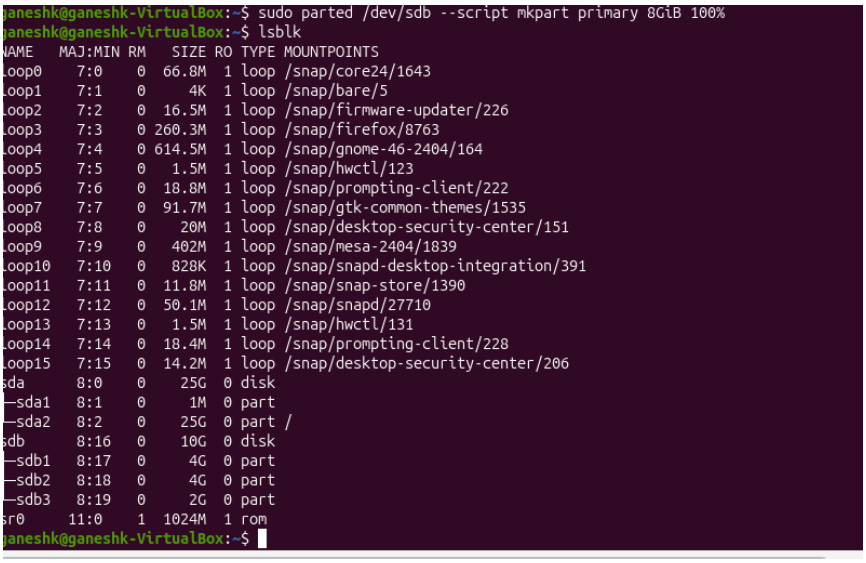


Figure 3.4: lsblk output showing the partitions sdb1, sdb2 and sdb3

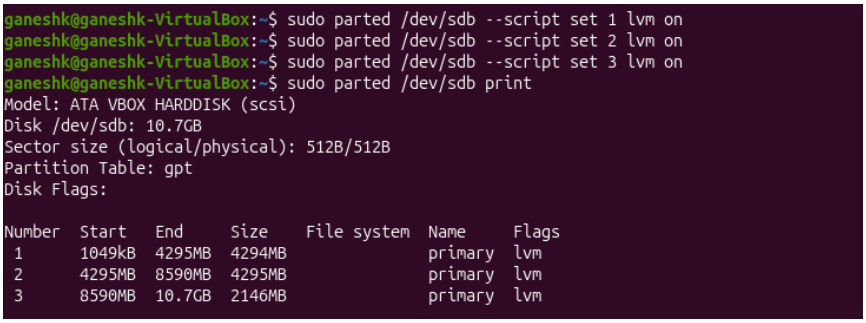


Figure 3.5: parted print: lvm flag set on all three partitions

3.6.2 LVM Configuration

Physical volumes were created on sdb1 and sdb2, and the volume group vg_storage was created from them. The third partition (sdb3) was added later with vgextend, so the pool grew to about 10 GB. Two logical volumes were then created: lv_data (300 MB) for files and lv_backup (200 MB) for backups.

```
sudo pvcreate /dev/sdb1 /dev/sdb2
sudo vgcreate vg_storage /dev/sdb1 /dev/sdb2
sudo lvcreate -L 300M -n lv_data vg_storage
sudo lvcreate -L 200M -n lv_backup vg_storage
sudo pvcreate /dev/sdb3
sudo vgextend vg_storage /dev/sdb3
```

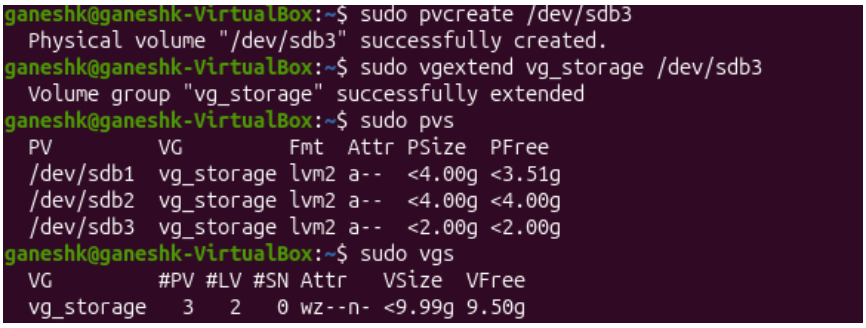


Figure 3.6: sdb3 added to vg_storage with vgextend (pvs and vgs output)

3.6.3 Filesystems and Mounting

An ext4 filesystem was created on each logical volume and the volumes were mounted. To keep the mounts after a reboot, two lines were added to /etc/fstab.

```
sudo mkfs.ext4 /dev/vg_storage/lv_data
sudo mkfs.ext4 /dev/vg_storage/lv_backup
sudo mkdir -p /mnt/data /mnt/backup
sudo mount /dev/vg_storage/lv_data /mnt/data
sudo mount /dev/vg_storage/lv_backup /mnt/backup
```

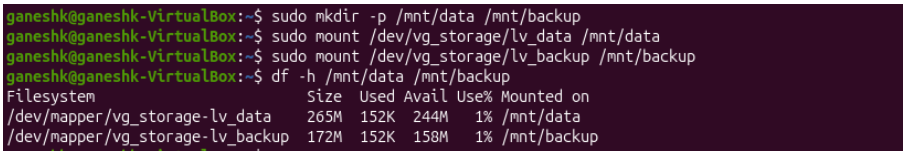


Figure 3.7: df -h output: both logical volumes are mounted

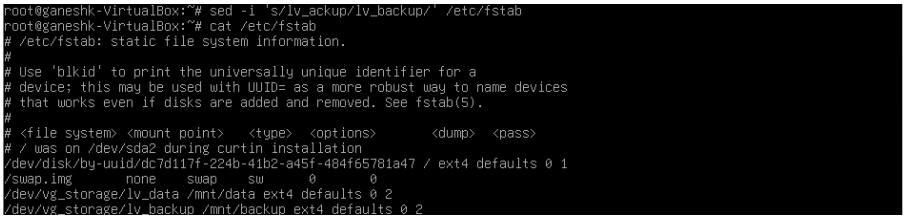


Figure 3.8: Permanent mount entries in /etc/fstab (last two lines)

3.6.4 Extending a Volume Online

The volume lv_data was extended from 300 MB to 500 MB while it was mounted. The option -r also resizes the ext4 filesystem, so no unmounting is needed.

```
sudo lvextend -L +200M -r /dev/vg_storage/lv_data
```

```
ganeshk@ganeshk-VirtualBox:~$ sudo lvextend -L +200M -r /dev/vg_storage/lv_data
  File system ext4 found on vg_storage/lv_data mounted at /mnt/data.
  Size of logical volume vg_storage/lv_data changed from 300.00 MiB (75 extents) to 500.00 MiB (125 extents).
  Extending file system ext4 to 500.00 MiB (524288000 bytes) on vg_storage/lv_data...
resize2fs /dev/vg_storage/lv_data
resize2fs 1.47.2 (1-Jan-2025)
Filesystem at /dev/vg_storage/lv_data is mounted on /mnt/data; on-line resizing required
old_desc_blocks = 1, new_desc_blocks = 1
The filesystem on /dev/vg_storage/lv_data is now 128000 (4k) blocks long.

resize2fs done
  Extended file system ext4 on vg_storage/lv_data.
  Logical volume vg_storage/lv_data successfully resized.
ganeshk@ganeshk-VirtualBox:~$ df -h /mnt/data
Filesystem                      Size  Used Avail Use% Mounted on
/dev/mapper/vg_storage-lv_data  459M  152K  434M   1% /mnt/data
ganeshk@ganeshk-VirtualBox:~$ sudo lvs
  LV        VG         Attr       LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv_backup vg_storage -wi-ao---- 200.00m
  lv_data   vg_storage -wi-ao---- 500.00m
```

Figure 3.9: lv_data extended online from 300 MB to 600 MB

3.6.5 Snapshot

A snapshot is a frozen copy of a volume at one moment. A test file was created, a snapshot was taken, the file was deleted, and the snapshot was mounted read-only to recover the file. The snapshot was removed afterwards.

```
sudo lvcreate -L 100M -s -n snap_data /dev/vg_storage/lv_data
sudo mount -o ro /dev/vg_storage/snap_data /mnt/snap
sudo lvremove -y /dev/vg_storage/snap_data
```

```
ganeshk@ganeshk-VirtualBox:~$ sudo lvs
  LV        VG         Attr       LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv_backup vg_storage -wi-ao---- 200.00m
  lv_data   vg_storage -wi-ao---- 500.00m
ganeshk@ganeshk-VirtualBox:~$ sudo sh -c 'echo "important data" > /mnt/data/test.txt'
ganeshk@ganeshk-VirtualBox:~$ cat /mnt/data/test.txt
important data
ganeshk@ganeshk-VirtualBox:~$ sudo lvcreate -L 100M -s -n snap_data /dev/vg_storage/lv_data
  Logical volume "snap_data" created.
ganeshk@ganeshk-VirtualBox:~$ sudo lvs
  LV        VG         Attr       LSize   Pool Origin  Data%  Meta%  Move Log Cpy%Sync Convert
  lv_backup vg_storage -wi-ao---- 200.00m
  lv_data   vg_storage owi-aos--- 500.00m
  snap_data vg_storage swi-a-s--- 100.00m      lv_data 0.01
ganeshk@ganeshk-VirtualBox:~$ sudo rm /mnt/data/test.txt
ganeshk@ganeshk-VirtualBox:~$ ls /mnt/data
lost+found
ganeshk@ganeshk-VirtualBox:~$ sudo mkdir -p /mnt/snap
ganeshk@ganeshk-VirtualBox:~$ sudo mount -o ro /dev/vg_storage/snap_data /mnt/snap
mount: (hint) your fstab has been modified, but systemd still uses
       the old version; use 'systemctl daemon-reload' to reload.
ganeshk@ganeshk-VirtualBox:~$ cat /mnt/snap/test.txt
important data
```

Figure 3.10: Deleted file recovered from the snapshot

3.7 Backup, Automation and Monitoring

3.7.1 Backup Script

The script backup.sh compresses the folder /mnt/data using tar and saves it in /mnt/backup. The date and time are part of the file name, and the result is written to /var/log/backup.log. The full script is given in Appendix A.

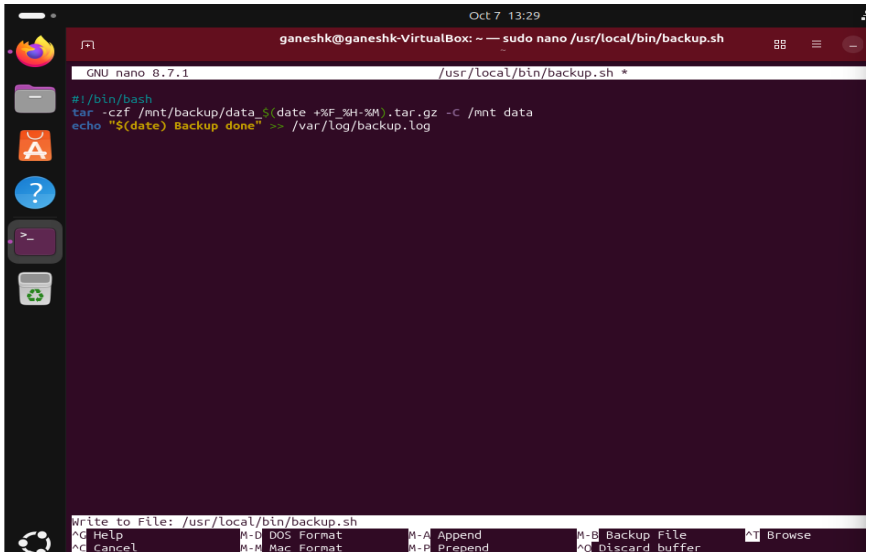


Figure 3.11: The backup script backup.sh

```
aneshk@ganeshk-VirtualBox:~$ ls -1 /mnt/data
ost+found
otes.txt
ecord.txt
est.txt
aneshk@ganeshk-VirtualBox:~$ sudo nano /usr/local/bin/backup.sh
sudo: authenticate] Password:
aneshk@ganeshk-VirtualBox:~$ sudo chmod +x /usr/local/bin/backup.sh
aneshk@ganeshk-VirtualBox:~$ cat /usr/local/bin/backup.sh

!/bin/bash
ar -czf /mnt/backup/data_$(date +%F_%H-%M).tar.gz -C /mnt data
cho "$(date) Backup done" >> /var/log/backup.log
```

Figure 3.12: Backup created in /mnt/backup

3.7.2 Restore Test

A backup is useful only if it can be restored. The archive was extracted into a different folder and the files were checked.

```
mkdir -p ~/restore_test
sudo tar -xzf /mnt/backup/FILENAME.tar.gz -C ~/restore_test
ls ~/restore_test/data
```

```
ganeshk@ganeshk-VirtualBox:~$ sudo /usr/local/bin/backup.sh
ganeshk@ganeshk-VirtualBox:~$ ls -lh /mnt/backup
total 20K
-rw-r--r-- 1 root root 265 Oct  7 13:36 data_2026-10-07_13-36.tar.gz
drwx------ 2 root root 16K Oct  7 12:24 lost+found
ganeshk@ganeshk-VirtualBox:~$ cat var/log/backup.log
cat: var/log/backup.log: No such file or directory
ganeshk@ganeshk-VirtualBox:~$ cat /var/log/backup.log
Wed Oct  7 01:36:58 PM IST 2026 Backup done
ganeshk@ganeshk-VirtualBox:~$ mkdir -p ~/restore_test
ganeshk@ganeshk-VirtualBox:~$ sudo tar -xzf /mnt/backup/data_*.tar.gz -C ~/restore_test
ganeshk@ganeshk-VirtualBox:~$ ls ~/restore_test/data
lost+found  notes.txt  record.txt  test.txt
ganeshk@ganeshk-VirtualBox:~$ cat ~/restore_test/data/test.txt
important data
```

Figure 3.13: Files restored from the backup

3.7.3 Automation with cron

Cron runs the scripts automatically. The backup runs every day at 2:00 AM and the monitoring script runs every 10 minutes.

```
0 2 * * * /usr/local/bin/backup.sh
*/10 * * * * /usr/local/bin/monitor.sh
```

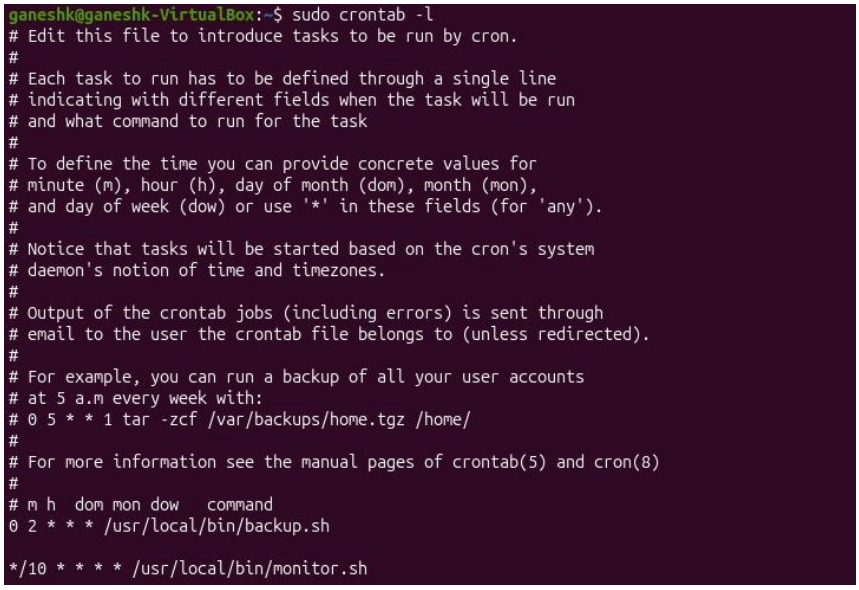


Figure 3.14: crontab -l showing the backup and monitoring jobs

3.7.4 Monitoring Script

The script monitor.sh writes the output of df, vgs and lvs to /var/log/storage_monitor.log. It reads the usage of /mnt/data and /mnt/backup and writes a WARNING when the usage is above 80%. The script was checked for syntax errors with bash -n. The full script is given in Appendix B.

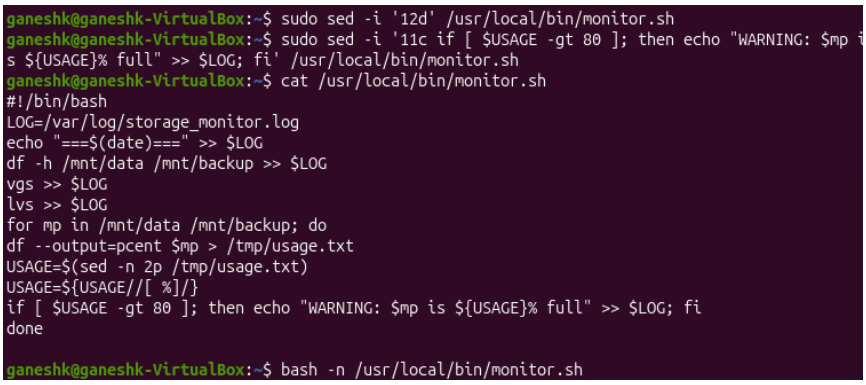


Figure 3.15: The monitoring script monitor.sh (final version)

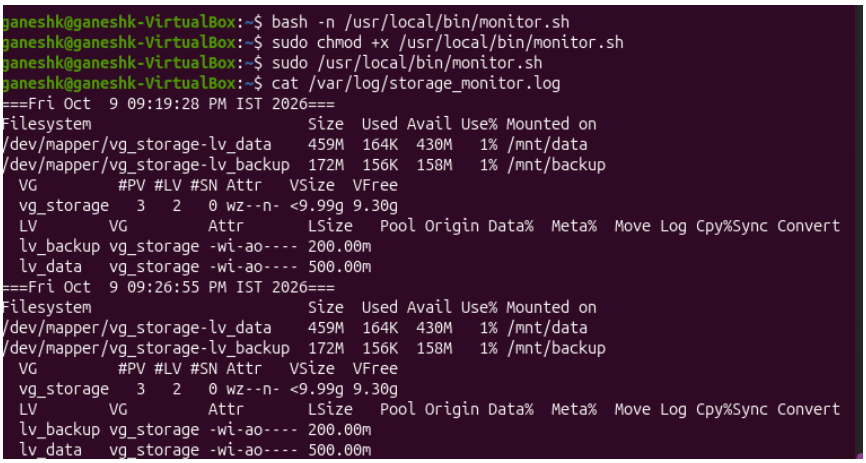


Figure 3.16: Monitoring log after running the script

To test the warning, a 150 MB file was created in /mnt/backup. The usage went to 96% and the script wrote a warning. The file was removed after the test.

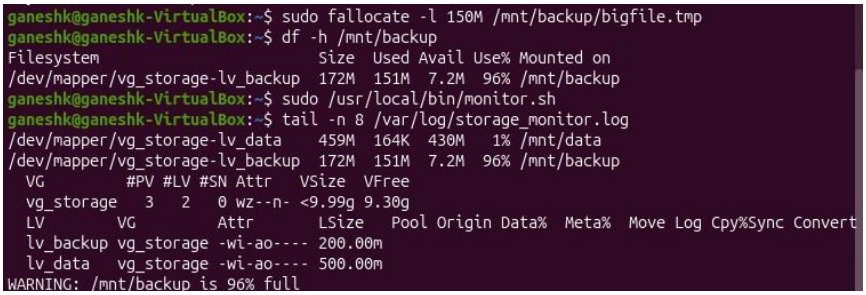


Figure 3.17: WARNING written when /mnt/backup reached 96%

3.8 Menu-Driven Interface

The script storage_manager.sh is the front end of the project. It shows a menu in the terminal, takes the choice of the user with the read command, runs the matching function, shows the result, and returns to the menu until the user selects Exit. It is started with:

```
sudo storage_manager.sh
```

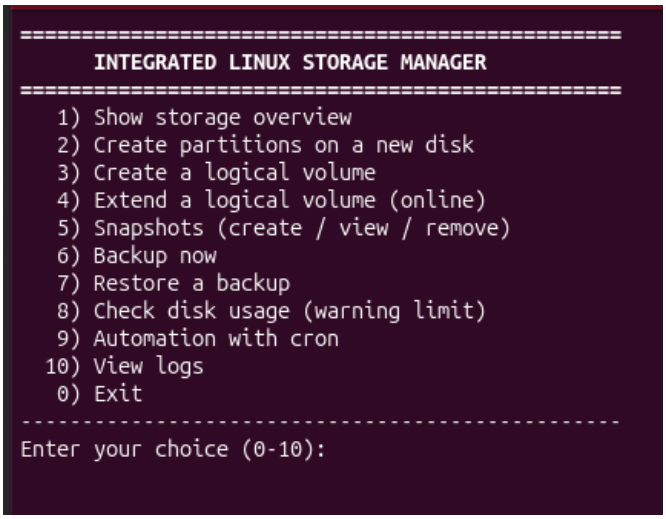


Figure 3.18: Main menu of the Integrated Linux Storage Manager

The menu options are:

Option	Function	Main commands used
1	Show storage overview	lsblk, pvs, vgs, lvs, df
2	Create partitions on a new disk	parted, pvcreate, vgextend
3	Create a logical volume	lvcreate, mkfs.ext4, mount
4	Extend a logical volume online	lvextend -r
5	Snapshots (create, view, remove)	lvcreate -s, mount -o ro, lvremove
6	Backup now	tar
7	Restore a backup	tar -x
8	Check disk usage with a warning limit	df, sed
9	Automation with cron	crontab
10	View logs	tail
0	Exit	-

Shell scripting concepts used

- **read -p:** takes input from the user.

- **while true loop:** keeps the menu running until Exit.
- **case:** selects the function for the chosen option.
- **functions:** one function for each menu option.
- **if and [[=~]]:** checks that inputs are valid numbers or names.
- **trap:** deletes temporary files when the script ends.
- **--backup and --monitor:** let cron run the backup and the usage check without showing the menu.

Error handling

The script checks the input of the user before doing any work:

Invalid input or error	Response of the script
Letters, symbols or empty input in the menu	Message: please enter a number
A number such as 99	Message: invalid choice
Wrong volume name or size	Input is rejected with an error message
Size larger than the free space in the volume group	Message that only X MB is free
Partitioning a disk that already has partitions	Refused, so the working disk is not erased
Partitioning the system disk	Refused
Backup folder missing or not writable	Error message and entry in the log
Script started without sudo	Message to run with sudo, then exit

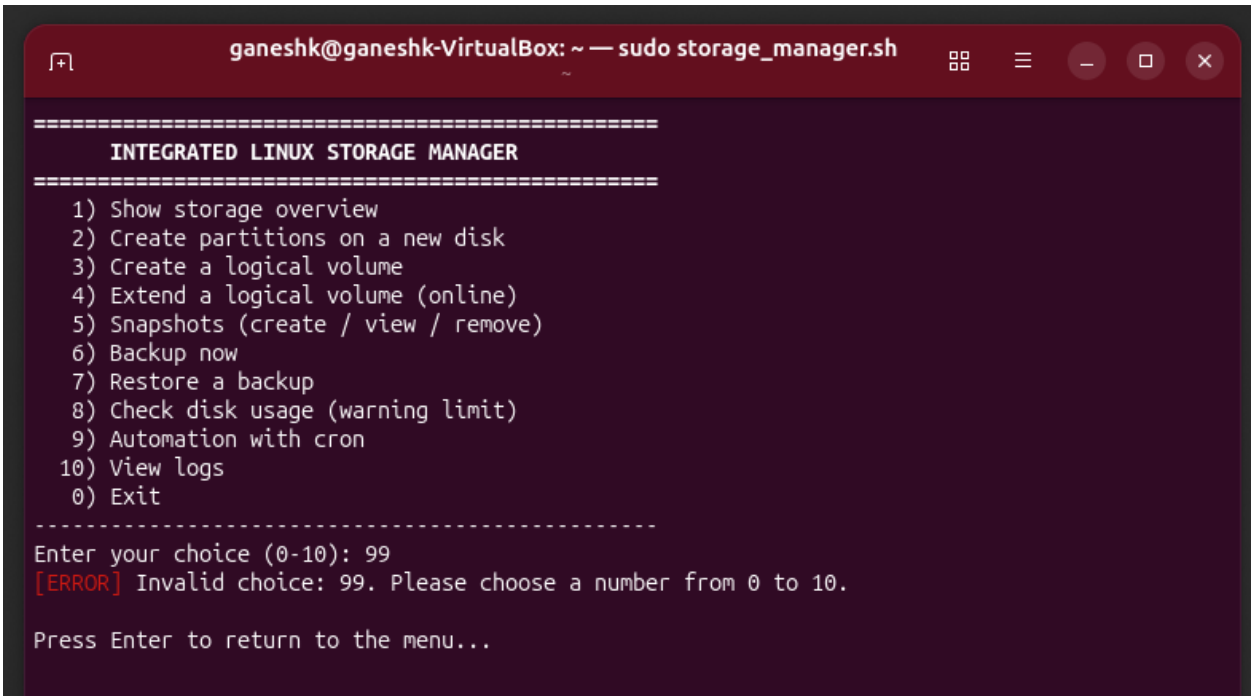


Figure 3.19: Error handling for invalid menu input

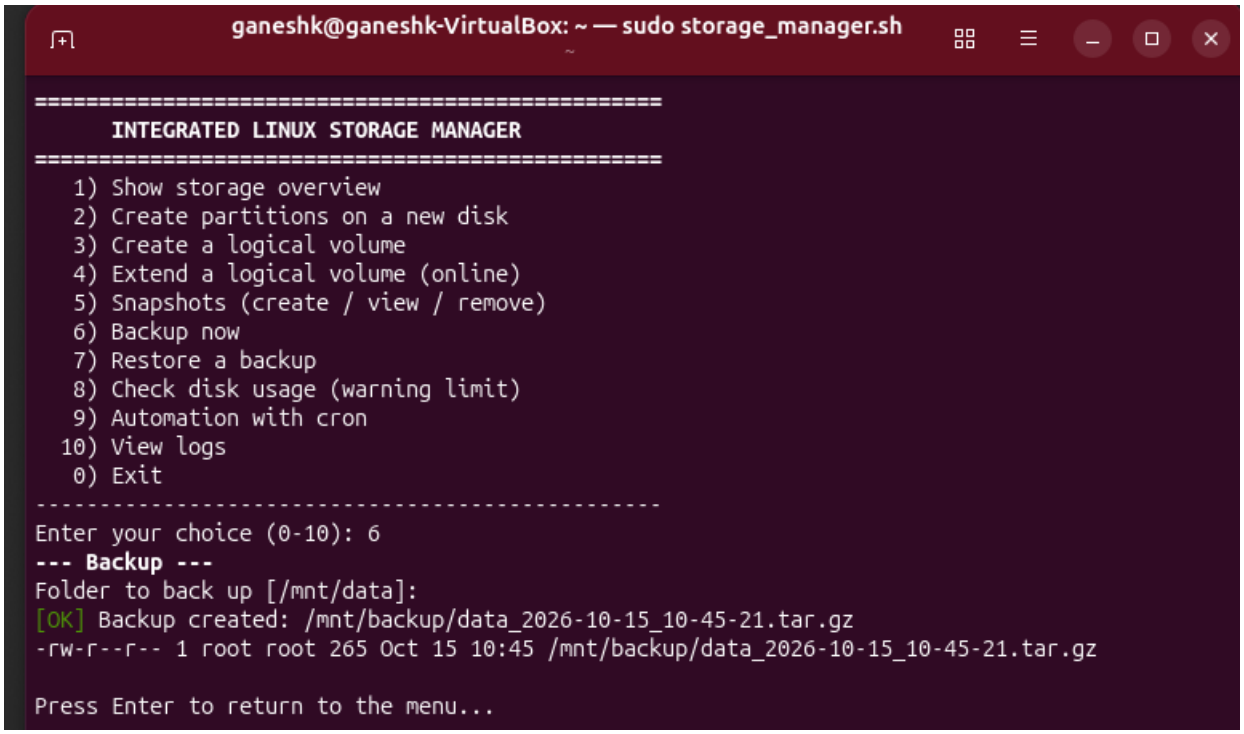


Figure 3.20: Output of a menu operation

3.9 Security and Access Control

- Storage commands run with sudo, so only authorised users can change disks.

- The scripts are stored in /usr/local/bin and made executable with chmod +x.
- Backups are kept on a separate volume from the original data.
- Snapshots are mounted read-only, so the saved copy cannot be changed.
- The menu refuses to partition the system disk or a disk that already has partitions.
- Only snapshot names starting with snap_ can be removed from the menu.
- New volumes added by the menu use the nofail option in /etc/fstab, so a mount problem does not stop the system from starting.
- All backup and monitoring actions are recorded in log files.

4. Results and Testing

4.1 Implementation Results

The complete infrastructure works as planned. After a reboot, both logical volumes were mounted again automatically because of the fstab entries, and the scripts were still present.

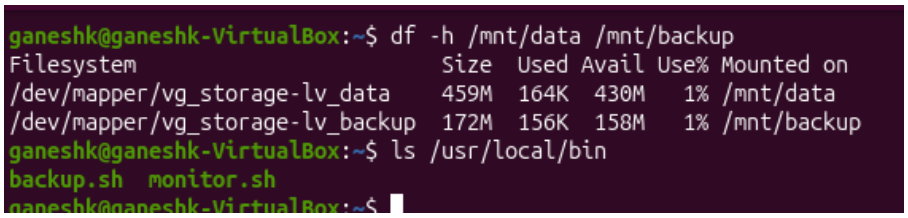


Figure 4.1: After reboot: both volumes are mounted and the scripts are present

The cron service ran the monitoring script by itself. A new entry with a fresh time stamp appeared in the log without being started by hand.

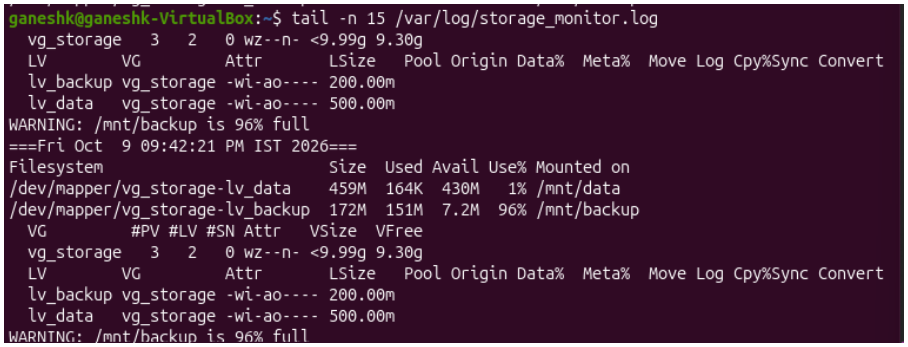


Figure 4.2: Log entry created automatically by cron

The final overview of the storage layers is shown below.

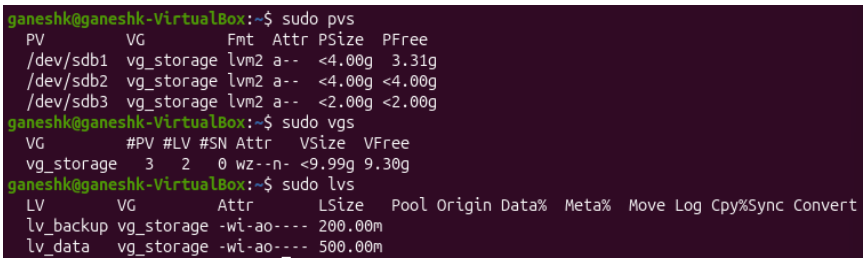


Figure 4.3: Final overview: pvs, vgs and lvs

4.2 Testing and Validation

Test	What was done	Result
Extend volume online	lv_data grown from 300 MB to 500 MB while mounted	Passed
Snapshot	Deleted a file and recovered it from the snapshot	Passed
Backup	tar.gz archive created in /mnt/backup	Passed
Restore	Archive extracted and the files were verified	Passed
Monitoring warning	/mnt/backup filled to 96% and a WARNING was written	Passed
Cron automation	monitor.sh ran automatically from cron	Passed
Reboot persistence	Volumes were mounted again after a reboot	Passed
Menu input checks	Letters, empty input and 99 were rejected	Passed
Menu operations	Overview, backup, restore, usage check and logs worked	Passed

4.3 Challenges Faced and Solutions

Problem	Cause	Solution
The VM started in emergency mode	Typing mistake in /etc/fstab: lv_ackup instead of lv_backup	Corrected with sed, tested with mount -a and rebooted
The | and \ keys did not work in the VM	Keyboard layout mismatch	Used commands and scripts that do not need these characters
monitor.sh gave a syntax error	Typing mistakes and a broken line	Found with bash -n and fixed with sed



Figure 4.4: The typing mistake lv_ackup in /etc/fstab that caused emergency mode

5. Conclusion and Future Scope

Conclusion

The project built and tested a complete Linux storage infrastructure using partitions, LVM, filesystems, backup, cron automation and monitoring. A menu-driven Bash script makes the whole system easy to operate and checks the input of the user. The system solves the problems of fixed storage, data loss, forgotten manual work and unnoticed full disks. All tests passed, including the reboot test.

Future Scope

- Add RAID (mdadm) so that one disk failure does not lose data.
- Send email or SMS alerts instead of only writing to a log.
- Copy backups to another server or to cloud storage.
- Encrypt the volumes using LUKS.
- Show usage graphs on a dashboard such as Grafana.
- Use LVM thin provisioning for more efficient use of space.

References

1. Canonical Ltd. Ubuntu Server Documentation: Logical Volume Management (LVM). documentation.ubuntu.com/server
2. Red Hat. Configuring and managing logical volumes. Red Hat Enterprise Linux documentation. docs.redhat.com
3. Linux man-pages: parted(8), lvm(8), mkfs.ext4(8), fstab(5), crontab(5), tar(1), df(1). man7.org
4. Oracle. Oracle VM VirtualBox User Manual. virtualbox.org
5. Free Software Foundation. GNU Bash Reference Manual. gnu.org/software/bash

Appendices

Appendix A: backup.sh

```
#!/bin/bash
tar -czf /mnt/backup/data_$(date +%F_%H-%M).tar.gz -C /mnt data
echo "$(date) Backup done" >> /var/log/backup.log
```

Appendix B: monitor.sh

```
#!/bin/bash
LOG=/var/log/storage_monitor.log
echo "===$(date)===" >> $LOG
df -h /mnt/data /mnt/backup >> $LOG
vgs >> $LOG
lvs >> $LOG
for mp in /mnt/data /mnt/backup; do
df --output=pcent $mp > /tmp/usage.txt
USAGE=$(sed -n 2p /tmp/usage.txt)
USAGE=${USAGE//[ %]/}
if [ $USAGE -gt 80 ]; then echo "WARNING: $mp is ${USAGE}% full" >> $LOG; fi
done
```

The full source code of storage_manager.sh (the menu-driven script) is submitted as a separate file.